

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки



Лабораторна робота
З дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав: ст.гр IX-22

Горячий Ростислав

Прийняв:

Гордійчук-Бублівська О.В

Львів – 2026

Мета: Розроблення клієнт-серверної архітектури з використанням REST API та практичне застосування класів для реалізації взаємодії між компонентами системи.

Розробити клієнт RestClient для взаємодії з REST API, реалізувавши методи GET та POST.

Вимоги до виконання:

- Реалізуйте клас RestClient, що підтримує такі методи:
- `get(endpoint)`: виконує HTTP GET-запит та повертає отримані дані.
- `post(endpoint, data)`: виконує HTTP POST-запит з переданими даними.
- Використовуйте бібліотеку `requests` для Python.
- Використайте відкритий REST API (наприклад, JSONPlaceholder або OpenWeatherMap), що дозволяє доступитися до розміщених там ресурсів. Для доступу до деяких API потрібно отримати API-ключ (див. інструкцію в теоретичній частині).
- Продемонструйте декілька прикладів використання RestClient для запитів до обраного API.
- Впевніться, що запити коректно обробляють відповіді сервера (перевірка статус-кодів, обробка помилок) та зробіть висновки

```
etpy
etpy > ...
1 import requests
2 import json
3 import os
4 class JsonOperations:
5     def __init__(self, filename: str):
6         self.filename = filename
7
8     def save_to_json(self, data):
9         with open(self.filename, 'w') as f:
10             json.dump(data, f, indent=4)
11
12     def read_json(self):
13         with open(self.filename, 'r') as f:
14             return json.load(f)
15
16 class RestWeatherClient:
17     def __init__(self, API_key):
18         self.API_key = API_key
19
20     def get_lat_lon(self, city, country_code):
21         url = f"http://api.openweathermap.org/geo/1.0/direct?q={city},{country_code}"
22         try:
23             res = requests.get(url)
24             res.raise_for_status()
25             data = res.json()
26
27             if len(data) == 0:
28                 print(f"City '{city}' not found")
29                 return None
30
31             return data[0]['lat'], data[0]['lon']
32
33         except requests.exceptions.RequestException as e:
34             print("Error getting coordinates:", e)
35             return None
36
37     def get_weather(self, coordinates: tuple):
38         if not coordinates:
39             return None
```

```

40
41     url = f"https://api.openweathermap.org/data/2.5/weather?lat={coordinates}"
42     try:
43         res = requests.get(url)
44         res.raise_for_status()
45         return res.json()
46
47     except requests.exceptions.RequestException as e:
48         print("Error getting weather:", e)
49         return None
50
51 class Weather:
52     def __init__(self):
53         self.info = {}
54
55     def basic_weather_info(self, full_weather):
56         if not full_weather:
57             return None
58
59         main = full_weather.get('main', {})
60         wind = full_weather.get('wind', {})
61
62         self.info = {
63             'temp': main.get('temp'),
64             'pressure': main.get('pressure'),
65             'humidity': main.get('humidity'),
66             'wind_speed': wind.get('speed')
67         }
68
69         return self.info
70
71 def process_city(rest_client, city, country_code, filename_prefix):
72     coordinates = rest_client.get_lat_lon(city, country_code)
73     if not coordinates:

```

```

74         return self.info
75
76 def process_city(rest_client, city, country_code, filename_prefix):
77     coordinates = rest_client.get_lat_lon(city, country_code)
78     if not coordinates:
79         return
80
81     weather_data = rest_client.get_weather(coordinates)
82     if not weather_data:
83         return
84
85     json_file = JsonOperations(f'{filename_prefix}.json')
86     json_file.save_to_json(weather_data)
87     base_info = json_file.read_json()
88     weather_base = Weather().basic_weather_info(base_info)
89     JsonOperations(f'base_{filename_prefix}.json').save_to_json(weather_base)
90
91 API_key = os.getenv("OPENWEATHER_API_KEY")
92
93 if not API_key:
94     print("API key not found. Set OPENWEATHER_API_KEY environment variable.")
95 else:
96     rest_client = RestWeatherClient(API_key)
97
98     Ukraine_code = 804
99     UK_code = 826
100
101     process_city(rest_client, 'Lviv', Ukraine_code, 'weather_Lviv')
102     process_city(rest_client, 'London', UK_code, 'weather_London')

```

```
{
  "coord": {
    "lon": -0.1278,
    "lat": 51.5074
  },
  "weather": [
    {
      "id": 804,
      "main": "Clouds",
      "description": "overcast clouds",
      "icon": "04n"
    }
  ],
  "base": "stations",
  "main": {
    "temp": 5.95,
    "feels_like": 2.96,
    "temp_min": 3.97,
    "temp_max": 6.88,
    "pressure": 999,
    "humidity": 72,
    "sea_level": 999,
    "grnd_level": 995
  },
  "visibility": 10000,
  "wind": {
    "speed": 4.12,
    "deg": 250
  },
  "clouds": {
    "all": 95
  },
  "dt": 1773430869,
  "sys": {
    "type": 2,
    "id": 2075535,
    "country": "GB",
```

```
    "sys": {
      "type": 2,
      "id": 2075535,
      "country": "GB",
      "sunrise": 1773382780,
      "sunset": 1773424835
    },
    "timezone": 0,
    "id": 2643743,
    "name": "London",
    "cod": 200
  }
}
```

```

import requests
import json
class JsonOperations:
    def __init__(self, filename: str):
        self.filename = filename

    def save_to_json(self, data):
        with open(self.filename, 'w') as f:
            json.dump(data, f, indent=4)

class RestClient:
    def __init__(self, url):
        self.url = url

    def _make_request(self, method, endpoint, **kwargs):
        try:
            res = requests.request(method, endpoint, **kwargs)
            res.raise_for_status()
            return res
        except requests.exceptions.RequestException as e:
            print(f"❌ Request error: {e}")
            return None

    def get(self, resource):
        endpoint = f"{self.url}/{resource}"
        res = self._make_request("GET", endpoint)

        if res:
            print("✅ GET OK")
            return res.json()

    def post(self, resource, data):
        endpoint = f"{self.url}/{resource}"
        res = self._make_request("POST", endpoint, json=data)

        if res:
            print("✅ Created:", res.json())

    def delete(self, resource, post_id):
        endpoint = f"{self.url}/{resource}/{post_id}"
        res = self._make_request("DELETE", endpoint)

```

```

40
41     if res:
42         print("✅ Post deleted")
43
44     def put(self, resource, post_id, new_data):
45         endpoint = f"{self.url}/{resource}/{post_id}"
46         res = self._make_request("PUT", endpoint, json=new_data)
47
48         if res:
49             print("✅ Post updated")
50
51
52     def main():
53         url = "https://jsonplaceholder.typicode.com"
54         client = RestClient(url)
55         posts = client.get('posts')
56         if posts:
57             JsonOperations('posts.json').save_to_json(posts)
58         data = {
59             "userId": 10,
60             "title": "test",
61             "body": "test body"
62         }
63
64         new_data = {
65             "userId": 10,
66             "title": "new_test",
67             "body": "new_test body"
68         }
69
70         client.post('posts', data)
71         client.put('posts', 1, new_data)
72         client.delete('posts', 1)
73

```

```

PS C:\Users\w1olf\Downloads\dwd> & C:/Users/w1olf/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/User
s/w1olf/Downloads/dwd/et.py
✅ GET OK
✅ Created: {'userId': 10, 'title': 'test', 'body': 'test body', 'id': 101}
✅ Post updated
✅ Post deleted

```

Висновок:

Під час виконання лабораторної роботи я спроектував клієнт-серверну архітектуру з використанням принципів **REST API**. Практична частина роботи дозволила закріпити навички використання об'єктно-орієнтованого підходу для реалізації взаємодії між окремими компонентами системи